

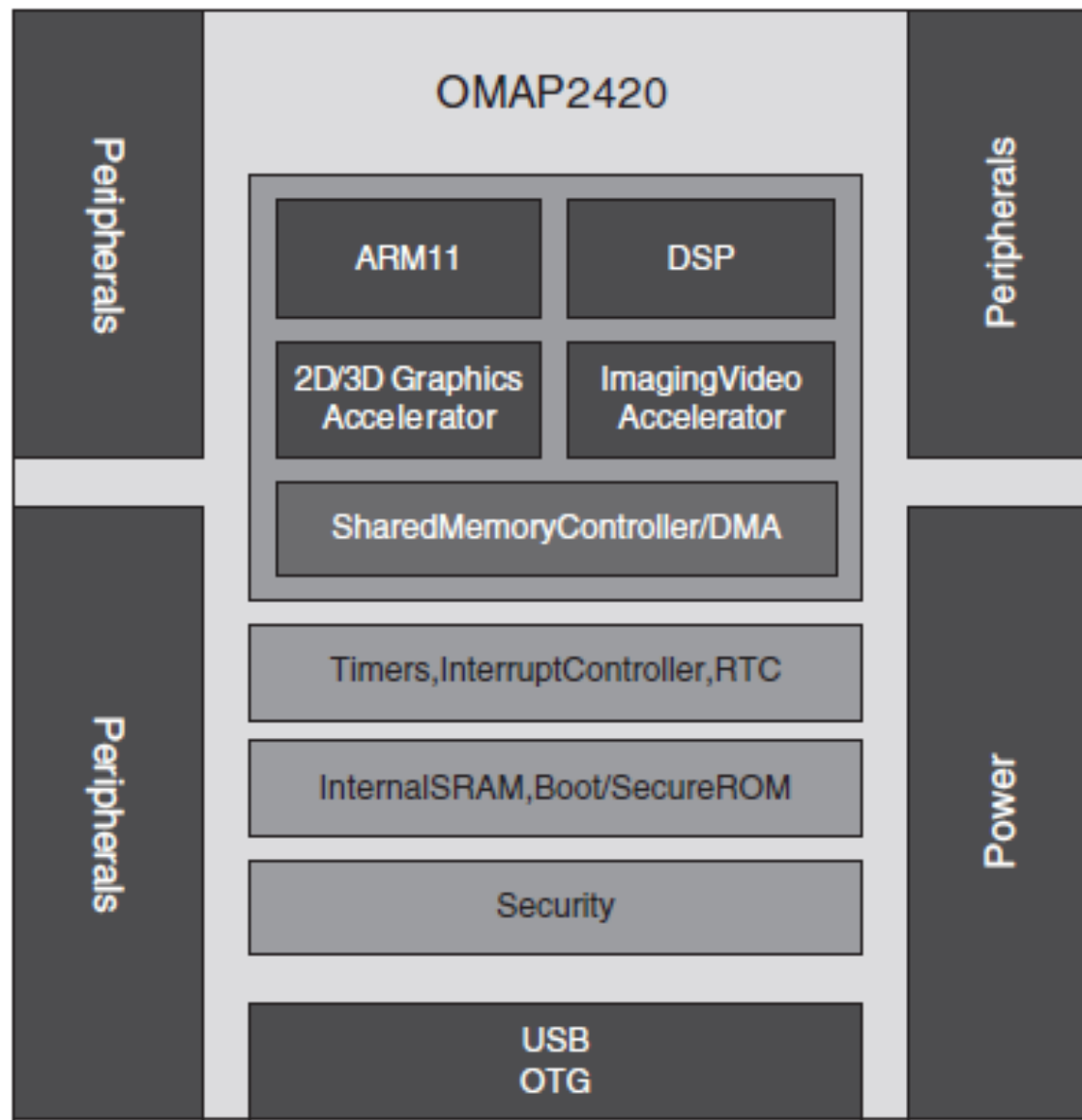
Functional Architecture Co Design

ESL: Electronic System Level

- The basic design component has evolved from transistors, to gates and then to register transfer level blocks.
- To cope with the challenges of designing emerging billion transistor system-on-chips (BTSOCs), it is widely believed that the chips have to be designed at an abstraction level well above RTL.
- Indeed, system designers typically reason about and conceptualize designs at an abstract functional level where system outputs are described as algorithms or transfer functions of the system inputs.
- This design abstraction, called electronic system level (ESL), enables ease of design capture and early design space exploration of multiple design implementation alternatives.
- ESL designs can be refined into lower levels of abstraction through a number of steps that gradually map abstract functions into register-transfer level (RTL) components, which is the next level of design abstraction.

ESL Design Methodology

- Although RTL design methods are dominant currently, rapid growth in chip complexity, coupled with shrinking time-to-market windows, results in RTL design methods not being able to scale with the complexity of emerging designs.
- This trend necessitates fundamental changes that force the move toward higher levels of abstraction. Let us examine two such fundamental changes.
 - The first fundamental change is that the cost of a new chip design by use of an RTL design method is no longer economically viable.
 - The second fundamental change is the rapid growth in chip complexity that enables integration of previously separate components on a printed circuit board (e.g., CPU, Ethernet controller, or memory) into a single chip. This integration of heterogeneous “content” comprises not only hardware but also software. Indeed, a chip is more often designed as a programmable computer system, requiring not only hardware but also firmware, operating systems, and application software, with applications often downloaded after the end product is deployed to consumers.



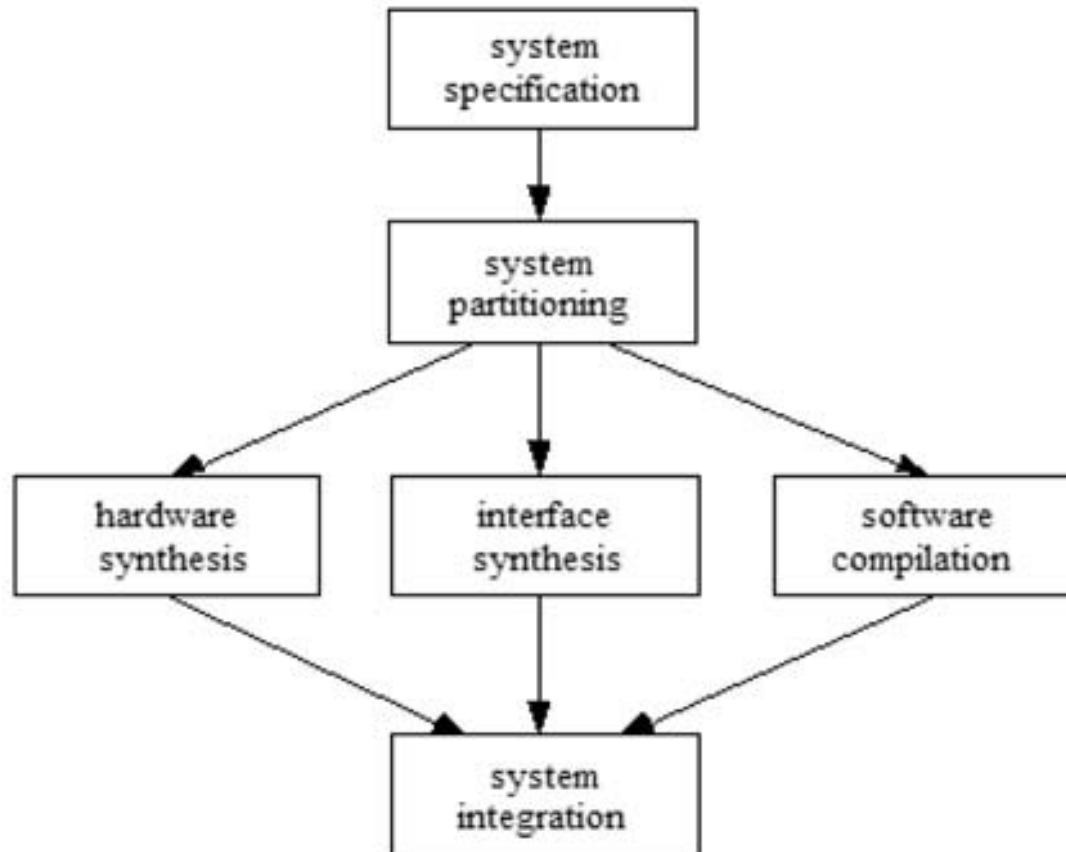
Texas Instrument's OMAP platform designed for cellular phone

HW/SW Co Design

- Computing systems are becoming increasingly complex and often contain large amounts of both hardware and software. The need for methodological support in managing the development process of such systems, therefore, becomes more urgent.
- Co-design refers to a methodology for the integrated design of systems implemented using both hardware and software components, given a set of performance goals and an implementation technology.
- The user's requirements are partitioned into functional and nonfunctional subsets, from which a functional view and an architectural view, i.e., the environment the system has to function in, can be derived.
- The subject of computing is then split between hardware and software at an early stage.
- The hardware development will be based on cost and performance, whilst the software development will be guided by the functional requirements of the system.

Cont...

- The co-design approach entails
 - Characterizing hardware and software performance.
 - Identifying a hardware/software partition.
 - Transforming the functional description into such a partition.
 - Synthesizing the resulting hardware and software.



The Co Design Flow

HW/SW Partitioning

- Given some basis for evaluating system performance, it is possible to decide which tasks should be implemented as hardware and which as software.
- If a task interacts closely with the operating system, software may be the only feasible implementation. Likewise, if a task interacts closely with external signals, implementing it in hardware may be the only practical solution.
- one can determine which to pursue according to the following criteria:
 - **Dynamic properties of the system:** a characterization of how the execution time of a task impacts system performance;
 - **Static properties of the task:** the difference in execution times between hardware and software implementations of the task;
 - **Hardware costs:** the amount of custom hardware required to realize a hardware implementation of the task.

Hardware Synthesis Advantages

- **Shorter design cycle**, which means that a product can be completed faster, decreasing the cost in design and the time-to-market.
- **Fewer errors**, since the synthesis process can be verified, increasing the chance that the final design will correspond to the initial specification.
- **The ability to search the design space**, since the synthesis system can offer a variety of solutions from the same specification, i.e., a particular solution can be chosen by the designer in order to satisfy tradeoffs between cost, speed, or power.
- **Documenting the design process**, which means keeping track of the decisions taken and their effects.

Examples

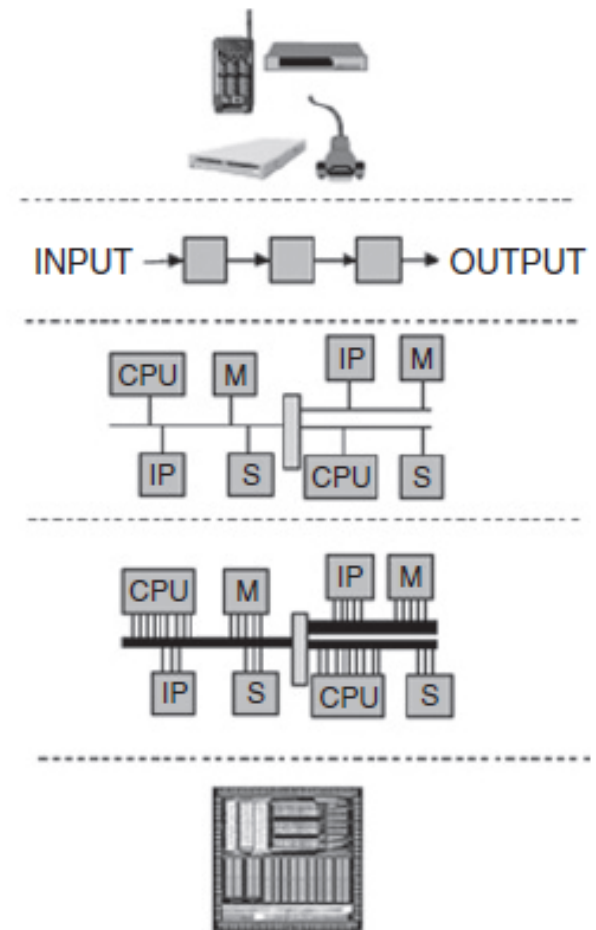
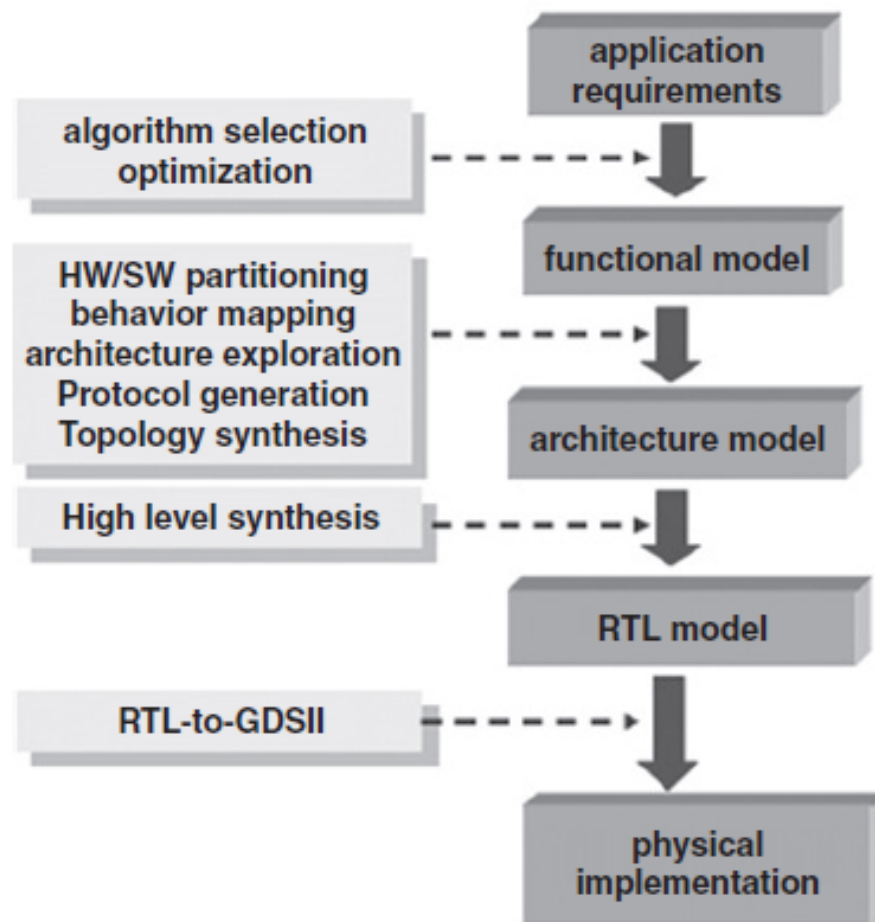
- An example of a framework for co-design is PTOLEMY (J. Buck and Messerschmitt, 1990; University of California and Science, 1992), a software environment for the simulation and prototyping of heterogeneous systems.
- Xilinx SDSoC platform, with Zynq boards supports HW/SW Co design.

Function architecture codesign methodology

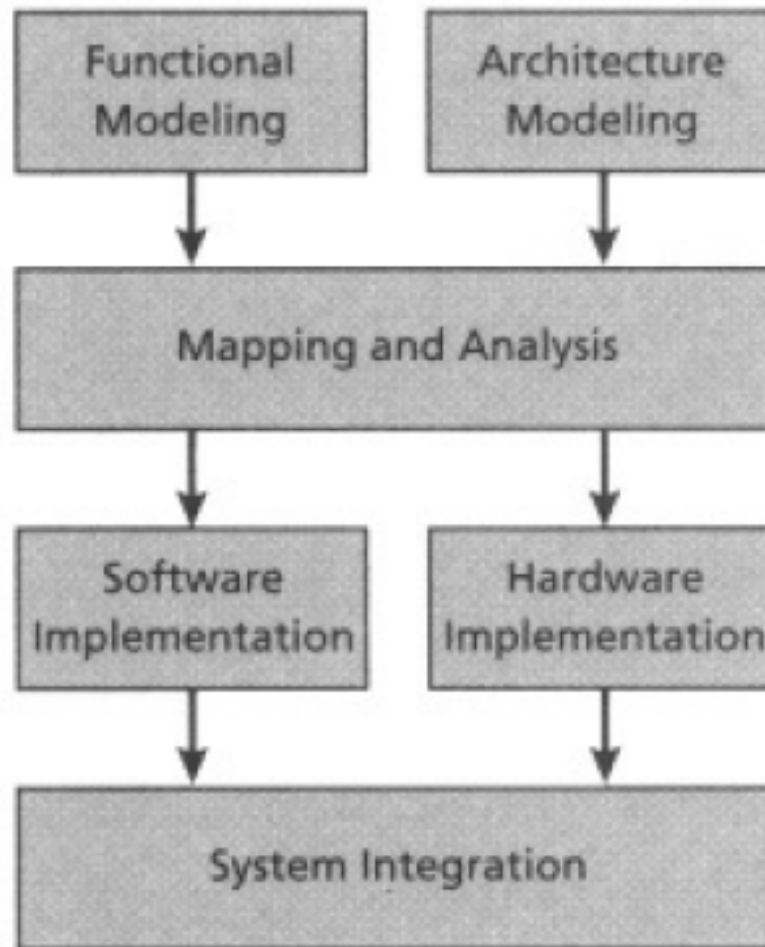
- This function-based ESL design method uses a computational model to compose different functional components into a complete system.
- The computational model determines how the components execute in parallel and how they communicate with each other.
- Function-architecture co-design begins at a level of design abstraction above that normally used in integrated circuit (IC) design, which today starts at register-transfer level (RTL) on the hardware side, and the C-code level on the software side.
- Function-architecture co-design begins with a purely functional model of the desired product behavior, and abstract models of system architecture suitable for performance evaluation.

Cont...

- This method follows a top-down, stepwise refinement approach in which designers start with design requirements followed by the development of a functional model.
- Here the functional model consists of a network of functional components under a specific computational model, thereby capturing the system function as a relation between system output and input.
- This functional model is gradually refined into an architectural model, which consists of a network of architectural components that communicate at the transaction level.
- The architectural model can be further refined into RTL by a step of high-level synthesis as described in the next section.
- Finally, the RTL model can be implemented in silicon by means of “RTL-handoff ” to the ensuing steps of logic synthesis and physical design.



Function Architecture Co Design Model



Phases Of Functional Architecture Co Design

Functional Modeling

- In this phase, the product requirements are established, and a verified specification of the system's function or behavior is produced.
- The specification can be executable, and it can also link to an executable specification of the environment within which the system will be embedded as a product.
- For example, a cellular handset might use a verification environment for a wireless standard such as GSM or IS-95 (CDMA).
- Functional exploration includes function and algorithm design and verification, probably simulation-based.
- The environmental executable specification can be considered a virtual testbench that can be applied to system-level design, as well as provide a verification testbench for implementing phases of the SOC design later.

Architecture Modeling

- Once a functional specification for a product has been developed, a candidate architecture or family of architectures on which the system functionality will be realized is defined.
- Hardware/software architectures include a variety of components, such as microprocessors and microcontrollers, digital signal processors (DSP), buses, memory components, peripherals, real-time operating systems (RTOS), and dedicated hardware processing units (for example, MPEG audio and video decoders).
- Such components can be reused VC blocks, whether in-house, third-party, or yet-to-be-designed blocks.
- The system functional specification is decomposed and mapped onto the architectural blocks. Possibilities for component blocks include reconfigurable hardware blocks that are one-time, dynamic, with long reconfiguration latency, or dynamic with a short enough reconfiguration latency for the block to offer multiple function modes with considerable area efficiency.

Mapping

- This process maps, or partitions, the functional model onto the architecture model by assigning every function to a specific hardware or software resource: for hardware, as a dedicated hardware block (or one mode of a dedicated hardware block); for software, as a task running on a general or specialized processor.
- Embedded systems that contain several processors offer several choices for mapping particular software functions. They require, at a minimum, a basic task scheduler, up to a full RTOS, to mediate access to each processor's resources.
- Although manual mapping can adequately deal with many systems, research into automated mapping algorithms might lead to methods that will be key to finding optimal mappings for the very complex embedded systems of the future.
- After mapping, various kinds of performance analysis are possible, which might lead to experiments involving alternative architectures or alternative choices of VCs before an optimal architecture and mapping are found. A certain amount of architectural refinement can also be carried out prior to proceeding to the implementation phases.

SW/HW implementation and System Integration

- This phase involves designing new hardware blocks, integrating reused hardware VC blocks, and developing software. Typical IC design today begins at this level of abstraction, often called the “RTL-C” level.
- With developed software and hardware, at least in prototype form, the complete system can be assembled for lab trials. Product integration might include emulators or rapid prototypes for hardware functions.

